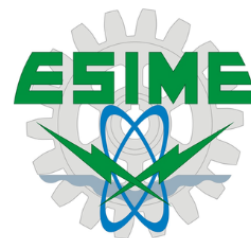

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE INGENIERIA
MECÁNICA Y ELÉCTRICA



INGENIERÍA EN COMUNICACIONES Y
ELECTRÓNICA



MATERIA:
SINCRONIZACIÓN Y MULTIPLEXAJE

PRACTICA 2
WEB APP EN ESP32 S3

DOCENTE:
LAURA HERNANDEZ HERRERA

ALUMNO:
-JUÁREZ SÁNCHEZ LUCIANO

BOLETA:
- 2021351629

Jueves, 04 de Febrero de 2026

Objetivo General

Implementar un sistema embebido IoT basado en el ESP32-S3 que permita el control sincrónico de dos cargas (focos) y el monitoreo ambiental mediante una interfaz web, aplicando los principios de sincronización y multiplexaje aprendidos en clase.

Objetivos Específicos

1. Diseñar y programar un servidor web embebido en el ESP32-S3 para servir una interfaz de usuario desde la cual controlar los focos y visualizar datos.
2. Configurar y utilizar el bus I2C para la comunicación multiplexada con el sensor AHT10, extrayendo datos de temperatura y humedad de forma periódica y no bloqueante.
3. Sincronizar el reloj interno del sistema con servidores NTP (Network Time Protocol) para obtener y mostrar la hora y fecha exactas.
4. Implementar la lógica de control que garantice la sincronización en tiempo real entre el estado de la interfaz web, las variables internas del programa y el estado físico de los relés.
5. Integrar todos los módulos (control web, sensor I2C, gestión de tiempo y salidas GPIO) en un único firmware que funcione de manera concurrente, estable y eficiente.

INTRODUCCIÓN

La presente práctica se enfoca en la aplicación concreta de los conceptos fundamentales de sincronización y multiplexaje en sistemas digitales, llevándolos al campo práctico del Internet de las Cosas (IoT). La sincronización se refiere a la coordinación precisa de eventos y procesos, mientras que el multiplexaje permite la transmisión eficiente de múltiples flujos de información a través de un canal compartido.

Para demostrar esto, se desarrolló un sistema de control y monitoreo ambiental usando la placa **ESP32-S3**, un microcontrolador con capacidades de conectividad Wi-Fi. El proyecto integra tres ejes principales: 1) el **control de salidas** (dos focos mediante relés), 2) la **adquisición de datos** (usando un sensor AHT10 conectado vía I2C), y 3) una **interfaz de usuario** (una aplicación web servida por el propio dispositivo).

El corazón del sistema reside en la gestión sincronizada y no bloqueante de estas tareas dentro del firmware, asegurando que el servidor web responda de inmediato, que el sensor se lea a intervalos regulares sin congelar la interfaz, y que el estado del sistema sea consistente en todas sus capas (hardware, software e interfaz de usuario). Esta integración sirve como un ejemplo tangible de cómo los principios teóricos de la asignatura son esenciales para crear sistemas embebidos modernos, eficientes y confiables.

MATERIALES Y TECNOLOGÍA

Materiales Físicos (Hardware):

1. **Placa de desarrollo ESP32-S3.**
2. **Módulo de 2 relés** (de 5V o 3.3V, compatible con nivel lógico LOW Trigger).
3. **Sensor de temperatura y humedad AHT10** con interfaz I2C.
4. **Protoboard y cables jumper** (macho-hembra y macho-macho) para las conexiones.
5. **Fuente de alimentación** para el ESP32 y los relés (puede ser un cable USB y una fuente externa para las cargas).
6. **Dos focos o lámparas** para ser controlados.

Tecnologías y Software:

1. **Entorno de Desarrollo:** Arduino IDE o PlatformIO con el *board support package* para el ESP32-S3.
2. **Librerías de Arduino (C++):**
 - `WebServer.h` y `WiFi.h` para la conectividad Wi-Fi y el servidor HTTP.
 - `Wire.h` para la comunicación I2C.
 - `Adafruit_AHTX0.h` para manejar el sensor.
 - `ESPmDNS.h` para acceder al dispositivo por un nombre de red (`myhome.local`).
3. **Lenguajes de Interfaz de Usuario:** HTML5, CSS3 y JavaScript (van integrados en el *sketch* de Arduino como un *String* largo).
4. **Protocolos de Comunicación:**
 - **I2C:** Protocolo de bus serial síncrono y multiplexado por dirección para comunicar el microcontrolador con el sensor.
 - **HTTP/1.1:** Protocolo de la capa de aplicación para la comunicación entre el navegador web (cliente) y el servidor embebido.
 - **Wi-Fi (802.11):** Para la conectividad a la red local.
 - **NTP:** Protocolo para la sincronización de relojes a través de internet.
 - **mDNS:** Protocolo para resolver nombres de host a direcciones IP en redes locales sin un servidor DNS dedicado.

Antecedentes

La sincronización en sistemas digitales y de comunicaciones se refiere al proceso de coordinar eventos o acciones entre distintos componentes para asegurar su correcto funcionamiento conjunto. En el contexto de los sistemas embebidos y el Internet de las Cosas (IoT), esta coordinación es fundamental, no solo a nivel de señales de reloj interno (sincronización a nivel de hardware y software), sino también en la secuencia lógica de operaciones y en la interacción con usuarios o otros sistemas a través de interfaces.

El multiplexaje, por su parte, es una técnica que permite la transmisión de múltiples señales o flujos de datos a través de un único canal o recurso compartido, optimizando su uso. Un ejemplo clásico en sistemas embebidos es el bus I2C, un protocolo de comunicación serial que, mediante direccionamiento, multiplexa el acceso a múltiples periféricos (como sensores) utilizando solo dos líneas físicas.

En el ámbito de las prácticas de laboratorio de esta asignatura, se busca aplicar estos conceptos fundamentales en proyectos tangibles. Tradicionalmente, las prácticas de sincronización se abordan con circuitos digitales básicos, sistemas cliente-servidor simples o manejo de temporizadores. Sin embargo, la evolución de los microcontroladores, como la familia ESP32 de Espressif, ha ampliado enormemente el horizonte. Estos dispositivos integran capacidades de procesamiento dual, conectividad Wi-Fi y Bluetooth, y una amplia interfaz de periféricos, convirtiéndose en plataformas ideales para implementar sistemas complejos donde la sincronización y el multiplexaje son clave.

Para esta práctica libre, se decidió llevar estos conceptos a un escenario aplicado de domótica básica y monitoreo ambiental. El control simultáneo (sincronizado) de múltiples actuadores (focos) y la adquisición periódica de datos de un sensor a través de un bus compartido (I2C) ejemplifican directamente los principios teóricos. Además, el desarrollo de una interfaz web embebida introduce la necesidad de sincronizar el estado interno del sistema con las solicitudes asíncronas de un usuario remoto, gestionando múltiples tareas de manera concurrente en un único dispositivo. Así, la práctica une la teoría de sincronización/multiplexaje con tecnologías modernas de IoT, proporcionando un antecedente práctico y contextualizado para los temas centrales de la asignatura.

PROBLEMA

El desarrollo de sistemas embebidos inteligentes integrados en redes, como una aplicación de domótica básica, plantea desafíos técnicos que son terreno fértil para aplicar los principios de sincronización y multiplexaje. El problema central de esta práctica consistió en diseñar e implementar un sistema capaz de gestionar **múltiples tareas concurrentes y asíncronas** de manera coordinada y eficiente, utilizando un único microcontrolador.

1. **Gestión Concurrente y Sincronización de Tareas:** El sistema debe atender de forma responsiva y sin bloqueos tres operaciones principales que compiten por el tiempo de CPU: (a) la operación de un servidor web para recibir comandos del usuario, (b) el monitoreo periódico y la lectura del sensor AHT10, y (c) el control de salidas digitales (focos). Sin una sincronización adecuada (por ejemplo, mediante el uso de delays no bloqueantes, temporizadores o mecanismos de interrupción), una tarea podría monopolizar el procesador, haciendo que el sistema parezca lento o no responda.
2. **Multiplexaje en el Bus de Comunicación:** El sensor AHT10 utiliza el protocolo I2C, un bus serial bidireccional que es un caso clásico de multiplexaje por dirección. El problema aquí es garantizar que la comunicación con el sensor (lectura de datos) se realice de manera confiable y sin conflictos sobre el mismo bus físico (líneas SDA y SCL), especialmente si en el futuro se agregaran más dispositivos I2C. Se requiere un manejo correcto del inicio, direccionamiento, lectura/escritura y finalización de la comunicación para evitar corrupción de datos.
3. **Coherencia y Sincronización del Estado:** La interfaz web (cliente) y el firmware del ESP32 (servidor) mantienen estados separados (ON/OFF de los focos, valores de temperatura/humedad). Es crucial que cualquier cambio de estado generado por el usuario en la web se refleje inmediatamente en los pines GPIO (sincronización acción-reacción) y, a su vez, que la interfaz web muestre siempre el estado actualizado del sistema y los valores del sensor. La falta de esta sincronización llevaría a una experiencia de usuario confusa y a un control impreciso.
4. **Integración de Tecnologías Heterogéneas:** El proyecto requiere la integración cohesiva de tecnologías de diferentes capas: hardware (GPIO, protocolo I2C), lógica de control embebida (firmware en C++/Arduino), y una interfaz de usuario (HTML/JavaScript/CSS). Sincronizar el flujo de información a través de todas estas capas, desde un clic en un botón en el navegador hasta el encendido de un foco físico y la actualización de la página con datos del sensor, constituye el problema de ingeniería principal a resolver.

SOLUCIÓN / METODOLOGÍA / TECNOLOGÍA

Para resolver las problemáticas planteadas, se desarrolló un sistema embebido basado en el microcontrolador **ESP32-S3**, que integró hardware, software de control y una interfaz de usuario web. La metodología siguió un enfoque modular y por capas, asegurando que cada subsistema manejara sus responsabilidades específicas mientras se coordinaba eficientemente con los demás a través de mecanismos de sincronización definidos.

1. Tecnologías y Hardware Empleado

- **Microcontrolador:** ESP32-S3, por su doble núcleo, conectividad Wi-Fi integrada y abundantes periféricos GPIO e I2C.
- **Interfaz de Usuario:** Servidor Web embebido (WebServer.h), sirviendo una página HTML/CSS/JavaScript responsiva y moderna.
- **Comunicación y Sincronización Externa:**
 - **Protocolo I2C:** Para el multiplexaje de datos con el sensor AHT10 sobre las líneas SDA (GPIO7) y SCL (GPIO6).
 - **Protocolo HTTP:** Para la comunicación entre el cliente web y el servidor embebido.
 - **Protocolo NTP (Network Time Protocol):** Para sincronizar el reloj interno del sistema con un servidor de tiempo en Internet (pool.ntp.org).
- **Hardware de Potencia y Sensado:**
 - **Módulo de 2 Relés:** Activado con nivel LOW (LOW Level Trigger), conectado a los GPIO 16 y 17.
 - **Sensor AHT10:** Para la lectura de temperatura y humedad ambiental, conectado al bus I2C.

2. Metodología de Implementación y Solución de los Problemas

a) Gestión Concurrente y Sincronización de Tareas (No-Blocking)

Se implementó un patrón de programación **no bloqueante** basado en el estado de la función `millis()`. Esto permitió que el bucle principal (`loop()`) atendiera concurrentemente las solicitudes web y la lectura del sensor sin detenciones.

- **Servidor Web:** La librería `WebServer` maneja las solicitudes HTTP de forma asíncrona dentro de `server.handleClient()`. No utiliza `delay()`.

- **Lectura del Sensor:** La función `updateSensor()` verifica si han transcurrido **5000 milisegundos** desde la última lectura (`if (millis() - lastRead < 5000) return;`). Solo entonces realiza una nueva comunicación I2C. Esto sincroniza el muestreo a un intervalo fijo sin interferir con la responsividad del servidor.
- **Control de Salidas:** La activación/desactivación de los relés se realiza de manera inmediata dentro de los manejadores de las rutas web (`/relay1/on`, etc.), garantizando una respuesta en tiempo real a la acción del usuario.

b) Multiplexaje en el Bus I2C

El bus I2C se configuró con los pines específicos del ESP32-S3. La comunicación con el sensor AHT10 se gestionó a través de la librería `Adafruit_AHTX0`, la cual abstrae el protocolo de bajo nivel.

- **Inicialización:** `Wire.begin(SDA_PIN, SCL_PIN)` y `aht.begin()` configuran el bus y verifican la presencia del sensor en su dirección I2C por defecto.
- **Multiplexaje por Dirección:** El bus I2C permite conectar múltiples dispositivos. El microcontrolador actúa como **Maestro**, direccionando específicamente al AHT10 (esclavo) para iniciar una lectura. Este método asegura que, aunque solo haya un sensor en esta práctica, el sistema está preparado para multiplexar más dispositivos en el mismo bus físico sin conflictos.

c) Coherencia y Sincronización del Estado del Sistema

Se estableció un modelo cliente-servidor con estados sincronizados mediante una API REST simple.

- **Estado en el Servidor (ESP32):** Las variables booleanas `relay1State` y `relay2State` siempre reflejan el estado físico real de los relés.
- **API de Sincronización:**
 - **Endpoints de Control:** `/relay1/on`, `/relay1/off`, etc., cambian el estado del relevador y la variable correspondiente de forma atómica.
 - **Endpoint de Consulta:** `/relay/status` devuelve un objeto JSON con ambos estados, permitiendo al cliente web sondear y sincronizar su interfaz periódicamente.
 - **Endpoints de Datos:** `/temp`, `/hum`, `/time`, `/date` proporcionan los valores más recientes del sensor y del reloj sincronizado.

- **Sincronización en el Cliente (JavaScript):** La interfaz web usa fetch() para enviar comandos y, mediante setInterval(), actualiza los datos ambientales (cada 1 segundo) y el estado de los botones de los relés (cada 2 segundos), manteniendo la coherencia visual.

d) Integración de Capas y Sincronización del Tiempo

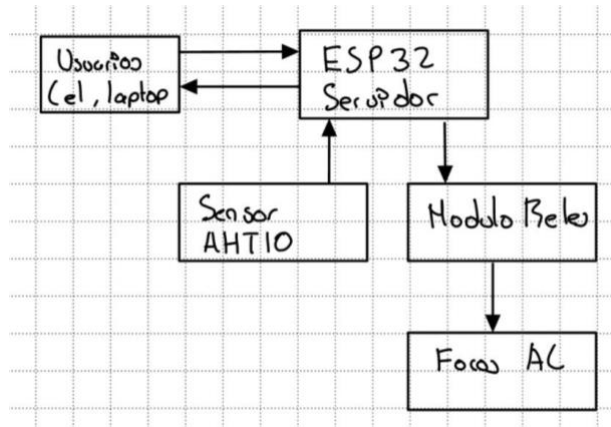
- **Sincronización del Reloj (NTP):** En el setup(), configTime() solicita la hora a un servidor NTP. Un bucle while(!getLocalTime(&timeinfo)) **bloquea** solo durante la inicialización hasta lograr la sincronización. Luego, las funciones getTimeString() y getDateString() proporcionan la hora sincronizada a la web bajo demanda.
- **Flujo de Datos Integral:** Un clic en un botón de la web → Solicitud HTTP → Manejador en el ESP32 → Cambio de estado del GPIO y variable → Respuesta HTTP → Actualización del botón vía JavaScript. Este flujo está sincronizado en cada paso, integrando HTML, lógica de red y hardware.

3. Estructura del Software (Firmware)

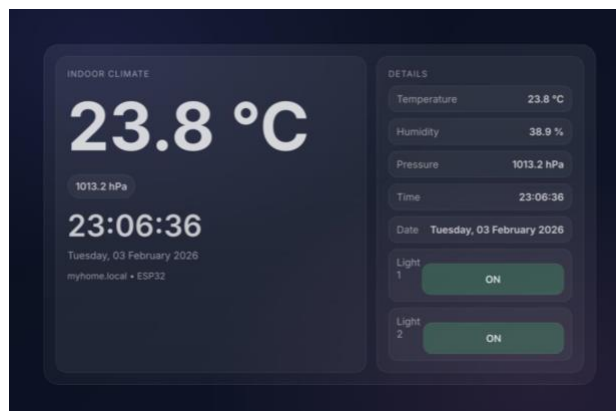
El código se organizó en secciones claras:

1. **Configuración:** Credenciales Wi-Fi, pines GPIO, inicialización de servidores y protocolos (I2C, NTP).
2. **Funciones de Soporte:** Para obtener tiempo, leer el sensor (con temporización no bloqueante) y formatear datos.
3. **Lógica Web:** La función handleRoot() sirve la página HTML completa. Se definieron múltiples manejadores de ruta (server.on()) para cada acción (control, estado, datos).
4. **Bucle Principal (loop()):** Ejecuta de manera continua y no bloqueante las dos tareas esenciales: atender peticiones web (server.handleClient()) y verificar si es momento de leer el sensor (updateSensor()).

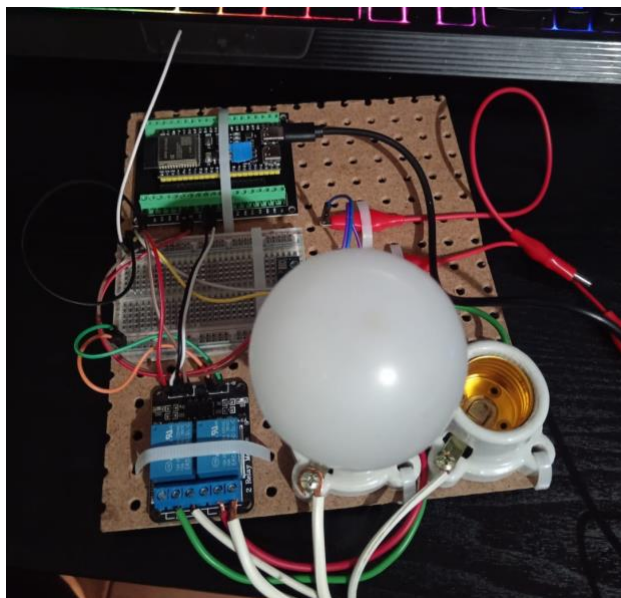
Diagrama a bloques



Web App



Circuito Físico



CONCLUSIÓN (Resultado)

El proyecto fue un éxito práctico y una gran experiencia de aprendizaje. Como resultado, funcionó un sistema de control domótico básico pero totalmente operativo desde cualquier dispositivo de la red local. Al acceder a myhome.local desde un navegador, se desplegaba una interfaz web profesional que mostraba en tiempo real la temperatura y humedad ambiente, la hora sincronizada desde internet, y dos botones para controlar los focos de manera independiente. El sistema respondía al instante a cada clic, y los datos del sensor se actualizaban periódicamente sin bloquear la interfaz.

BIBLIOGRAFÍA

- Espressif Systems. (s.f.). *Inter-Integrated Circuit (I2C) - ESP32*. ESP-IDF Programming Guide. Recuperado 5 de febrero de 2026, de <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/i2c.html>[citation:6]
- Random Nerd Tutorials. (s.f.). *ESP32 Async Web Server – Control Outputs with Arduino IDE (ESPAsyncWebServer library)*. Recuperado 5 de febrero de 2026, de <https://randomnerdtutorials.com/esp32-async-web-server-espasyncwebserver-library/>[citation:2]
- Arduino Forum. (2022, 28 julio). *AHT10 i2C Readings way off - Sensors - Arduino Forum* [Mensaje de foro]. Recuperado 5 de febrero de 2026, de <https://forum.arduino.cc/t/aht10-i2c-readings-way-off/1363981>[citation:1]
- Programar Fácil. (s.f.). *Servidor web con ESP32*. Recuperado 5 de febrero de 2026, de <https://programarfácil.com/esp32/servidor-web-con-esp32/>[citation:9]
- TodoMaker. (s.f.). *Montar un servidor web en el ESP32 con ESPAsyncWebServer y SPIFFS*. Recuperado 5 de febrero de 2026, de <https://todomaker.com/blog/montar-un-servidor-web-en-el-esp32-con-espasyncwebserver-y-spiiffs/>[citation:5]